

SKILL SESSION 19&20

Session 19 :

TASK 1: Implement how to register signal handlers, configure signal actions, handle interrupts, manage signal context, verify stability, and test signal events.

```
[(base) saiabburi@Charishma-MacBook-Air ~ % mkdir Skill10  
[(base) saiabburi@Charishma-MacBook-Air ~ % cd Skill10  
[(base) saiabburi@Charishma-MacBook-Air Skill10 % nano task1.c
```

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

volatile sig_atomic_t signal_received = 0;

void handle_signal(int sig)
{
    if (sig == SIGUSR1)
        write(STDOUT_FILENO, "SIGUSR1 received\n", 17);

    if (sig == SIGINT)
        write(STDOUT_FILENO, "SIGINT received - exiting\n", 26);

    signal_received = sig;
}

int main()
{
    struct sigaction sa;
    sigset_t set, pending;

    sa.sa_handler = handle_signal;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGUSR1, &sa, NULL);
    sigaction(SIGINT, &sa, NULL);

    sigemptyset(&set);
    sigaddset(&set, SIGUSR1);

    sigprocmask(SIG_BLOCK, &set, NULL);

    printf("Task 1: Signal handler registered.\n");
    printf("PID: %d\n", getpid());
    printf("SIGUSR1 is blocked for 5 seconds.\n");
    printf("Send SIGUSR1 using: kill -USR1 %d\n", getpid());

    sleep(5);

    sigpending(&pending);

    if (sigismember(&pending, SIGUSR1))
        printf("SIGUSR1 is pending.\n");

    printf("Unblocking SIGUSR1...\n");
```

```
sigprocmask(SIG_UNBLOCK, &set, NULL);

sleep(2);

printf("Signal handling test completed.\n");
printf("Press Ctrl+C to test SIGINT.\n");

while (1)
    sleep(1);

return 0;
}
```

```
[(base) saiabburi@Charishma-MacBook-Air Skill10 % gcc task1.c -o task1
[(base) saiabburi@Charishma-MacBook-Air Skill10 % ./task1
Task 1: Signal handler registered.
PID: 6790
SIGUSR1 is blocked for 5 seconds.
Send SIGUSR1 using: kill -USR1 6790
Unblocking SIGUSR1...
Signal handling test completed.
Press Ctrl+C to test SIGINT.
^CSIGINT received - exiting
SIGUSR1 received
^CSIGINT received - exiting
```

TASK 2: Implement how to capture SIGINT, forward signals, protect the shell process, terminate foreground jobs, update job states, and test interrupt handling.

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t child_pid;

void handle_sigint(int sig)
{
    printf("\nParent received SIGINT. Forwarding to foreground job...\n");
    if (child_pid > 0)
        kill(-child_pid, SIGINT);
}

int main()
{
    struct sigaction sa;
    int status;

    sa.sa_handler = handle_sigint;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, NULL);

    child_pid = fork();

    if (child_pid == 0)
    {
        setpgid(0, 0);

        printf("Foreground job started. PID: %d\n", getpid());
        printf("Press Ctrl+C to interrupt the foreground job.\n");

        while (1)
            sleep(1);
    }
    else
    {
        setpgid(child_pid, child_pid);

        waitpid(child_pid, &status, 0);

        if (WIFSIGNALED(status))
            printf("Foreground job terminated by signal %d.\n",
                WTERMSIG(status));
    }
}
```

```

        sleep(1);
    }
    else
    {
        setpgid(child_pid, child_pid);

        waitpid(child_pid, &status, 0);

        if (WIFSIGNALED(status))
            printf("Foreground job terminated by signal %d.\n",
                WTERMSIG(status));

        printf("Parent process is still running.\n");
        printf("Task 2 completed.\n");
    }

    return 0;
}

```

```

[(base) saiabburi@Charishma-MacBook-Air Skill10 % nano task2.c
[(base) saiabburi@Charishma-MacBook-Air Skill10 % gcc task2.c -o task2
[(base) saiabburi@Charishma-MacBook-Air Skill10 % ./task2
Foreground job started. PID: 6869
Press Ctrl+C to interrupt the foreground job.
^C
Parent received SIGINT. Forwarding to foreground job...
Foreground job terminated by signal 1.
Parent process is still running.
Task 2 completed.

Parent received SIGINT. Forwarding to foreground job...
(base) saiabburi@Charishma-MacBook-Air Skill10 %

```

Session 20 :

TASK 3: Implement how to capture SIGTSTP, suspend processes, update the job table, preserve process state, resume later, and test the suspension workflow.

```
[(base) saiabburi@Charishma-MacBook-Air Skill10 % nano task3.c
[(base) saiabburi@Charishma-MacBook-Air Skill10 % gcc task3.c -o task3
[(base) saiabburi@Charishma-MacBook-Air Skill10 % ./task3
Foreground process started. PID: 6916
Press Ctrl+Z to suspend it.
^Z
Parent received SIGTSTP. Suspending foreground process...

Parent received SIGTSTP. Suspending foreground process...
(base) saiabburi@Charishma-MacBook-Air Skill10 %
```

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t child_pid;

void handle_sigtstp(int sig)
{
    printf("\nParent received SIGTSTP. Suspending foreground process...\n");

    if (child_pid > 0)
        kill(-child_pid, SIGTSTP);
}

int main()
{
    struct sigaction sa;
    int status;

    sa.sa_handler = handle_sigtstp;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGTSTP, &sa, NULL);

    child_pid = fork();

    if (child_pid == 0)
    {
        setpgid(0, 0);

        printf("Foreground process started. PID: %d\n", getpid());
        printf("Press Ctrl+Z to suspend it.\n");

        while (1)
            sleep(1);
    }
    else
    {
        setpgid(child_pid, child_pid);

        waitpid(child_pid, &status, WUNTRACED);
    }
}
```

```

    if (WIFSTOPPED(status))
    {
        printf("Process suspended by signal %d.\n",
               WSTOPSIG(status));

        printf("Process state preserved.\n");
        printf("Resuming process after 5 seconds...\n");

        sleep(5);

        kill(-child_pid, SIGCONT);

        waitpid(child_pid, &status, 0);

        printf("Process resumed successfully.\n");
        printf("Task 3 completed.\n");

        kill(child_pid, SIGTERM);
    }
}

return 0;
}

```

TASK 4: Implement how to create process groups, assign group IDs, transfer terminal control, restore terminal ownership, coordinate signals, and validate process-group behavior.

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

int main()
{
    pid_t child_pid;
    pid_t original_pgid;
    int status;

    original_pgid = tcgetpgrp(STDIN_FILENO);

    signal(SIGTTOU, SIG_IGN);
    signal(SIGTTIN, SIG_IGN);
    signal(SIGTSTP, SIG_IGN);

    printf("Original terminal process group: %d\n",
           original_pgid);

    child_pid = fork();

    if (child_pid == 0)
    {
        setpgid(0, 0);

        printf("Child PID: %d\n", getpid());
        printf("Child Process Group ID: %d\n", getpgrp());

        sleep(10);

        return 0;
    }

    setpgid(child_pid, child_pid);
```

```

    printf("Created process group: %d\n", child_pid);

    sleep(1);

    printf("Transferring terminal control to child group...\n");

    tcsetpgrp(STDIN_FILENO, child_pid);

    printf("Child group now owns the terminal.\n");
    printf("Child process is running...\n");

    waitpid(child_pid, &status, 0);

    printf("Child completed.\n");
    printf("Restoring terminal control...\n");

    tcsetpgrp(STDIN_FILENO, original_pgid);

    printf("Terminal ownership restored.\n");
    printf("Task 4 completed.\n");

    return 0;
}

```

```

(base) saiabburi@Charishma-MacBook-Air Skill10 % nano task4.c
(base) saiabburi@Charishma-MacBook-Air Skill10 % gcc task4.c -o task4
(base) saiabburi@Charishma-MacBook-Air Skill10 % ./task4
Original terminal process group: 6941
Created process group: 6942
Child PID: 6942
Child Process Group ID: 6942
Transferring terminal control to child group...
Child group now owns the terminal.
Child process is running...
Child completed.
Restoring terminal control...
Terminal ownership restored.
Task 4 completed.
(base) saiabburi@Charishma-MacBook-Air Skill10 %

```